Name: Devansh Nayak
NUID:003181952

Classical Search Algorithms Project

Introduction

Pathfinding on graphs is one of the standard testbeds for classical artificial intelligence search. In this project we study deterministic grid navigation: an agent moves on a two-dimensional map from a fixed start cell to a goal cell using four-way moves with uniform step cost, while avoiding blocked cells. Because the state space grows quickly with grid size and obstacle patterns, practical systems rarely assume unlimited time or memory. Instead, real search pipelines must respect depth limits, expansion budgets, or wall-clock timeouts. Evaluating how different search strategies behave under those limits is therefore not only an implementation exercise but also an empirical question about robustness and efficiency.

Breadth-first search, depth-first search, iterative deepening depth-first search, and A* represent a representative slice of classical uninformed and informed methods. On unit-cost grids, breadth-first search and iterative deepening can recover optimal path length, whereas depth-first search may return feasible paths that are not optimal. A* uses a heuristic estimate of remaining cost and, under well-known conditions, can find optimal solutions with fewer expansions than uninformed breadth-first exploration. These relationships are textbook material, but they are usually presented for unconstrained search. This project extends the comparison to explicitly resource-bounded settings, where algorithms may terminate early and must report why they stopped.

The theoretical foundation for the informed-search component is the classical A* formulation of Hart et al. [1], which formalizes best-first search with an evaluation function that combines known path cost from the start with a heuristic estimate of cost-to-go. Their results about admissibility and consistency give a precise vocabulary for discussing when A* is guaranteed to return an optimal path and when stronger or weaker heuristics change the explored portion of the state space. Our empirical setup instantiates the usual implicit-graph view of search: states are grid cells, the successor relation generates legal moves, and arc costs are positive (here, uniformly one). We use breadth-first search on these instances as an optimal-cost reference for comparing A* outcomes when heuristics are admissible, and we study how heuristic scaling moves the algorithm away from optimality when the estimate can overstate remaining distance.

The remainder of this report describes the implemented environment and algorithms, explains how experiments are run and measured, and presents results that connect classical performance differences to resource constraints. The goal is a coherent account of what was built, how it was evaluated, and what the measurements imply for choosing and configuring search methods under realistic limits.

METHODS

This work addresses deterministic grid pathfinding: an agent moves from a start cell to a goal on a two-dimensional occupancy map with four orthogonal moves (up, down, left, right) and a uniform step cost of one, so path cost equals path length. The environment is implemented as a grid with traversable and blocked cells; optional random layouts are built with controlled obstacle density and a pseudo-random seed so experiments can be repeated. Search operates on explicit cell states; successors expand only in-bounds, non-obstacle directions.

The implementation portfolio follows classical textbook search: breadth-first search (BFS) as a completeness and optimality reference for uniform costs, depth-first search (DFS), iterative deepening depth-first search (IDDFS), and A* with injectable heuristics. The production A* module used in broad comparisons applies an admissible Manhattan distance to the goal. For theory-oriented analysis, a dedicated A* routine follows the standard formulation of ordered search with an explicit closed set (expanded nodes), an open list realized as a min-heap on the usual sum of estimated path cost from the start (often denoted g-hat) plus heuristic estimate to the goal (h-hat), and support for reopening a state when a better g-hat is discovered—behavior that matters when the heuristic is not consistent [1]. Along each transition, the code optionally checks the consistency (or “triangle”) condition: the heuristic at the parent should not exceed the step cost plus the

heuristic at the child; violations are counted for diagnosis. Heuristic variants include zero (uniform-cost search inside the A* shell), admissible Manhattan distance, a scaled Manhattan family with multiplicative weights in {1.0, 1.2, 1.5, 2.0, 3.0} to study intentional loss of admissibility, and a custom admissible-but-inconsistent heuristic designed to trigger consistency violations and reopenings without relying on external puzzle corpora.

Evaluation has two layers. First, all algorithms are run on the same grids under optional resource limits—maximum search depth, maximum node expansions, and wall-clock time budget—so truncated or stressed runs can be compared fairly. Metrics recorded include whether a solution was found, path cost, nodes expanded, wall time, and when applicable the reason a run stopped (for example, time or expansion limit). Second, a theory experiment batch pairs each grid with BFS as the optimal-cost baseline (when BFS succeeds under uniform cost), then runs A* variants with the heuristics above; this yields per-run flags for whether the returned path cost matches the baseline within a small numerical tolerance, the

suboptimality gap when it does not, counts of consistency violations detected during expansion, and counts of node reopenings. A larger sweep repeats this procedure over multiple grid sizes, obstacle densities, and seeds; results are aggregated into comma-separated tables, including final_theory.csv and sweep_results.csv used for plotting. Correctness is guarded by automated tests executed with the Python standard library unittest driver supplied in run_tests.py.

Reproducibility is supported by the fixed repository layout, seeded random grid generation, and command-line entry points that emit plain text or JSON for downstream tables. The bundled requirements.txt records optional developer dependencies installed with pip (e.g., pytest as an alternate test runner); the workflows reported here rely on unittest rather than pytest. Figure scripts read the exported comma-separated experiment tables and render publication-style plots from that pipeline.

No third-party pathfinding benchmark suites or proprietary maps were imported; every grid is either a fixed layout in the repository or generated by the project's own procedures. The algorithmic target of the study is the classical A* family described by Hart, Nilsson, and Raphael [1]. Execution and standard-library unittest-based verification use Python 3 [2]. Post-processing and figures use pandas [3], matplotlib [4], and seaborn [5], obtained with pip as needed alongside other listed dependencies.

RESULTS

The outcomes below correspond to the same evaluation design as in Methods section: (i) theory batches that compare BFS to theory-mode A* variants on seeded random grids (logged in final_theory.csv), and (ii) a multi-algorithm sweep over grids, obstacle densities, and resource limits (logged in sweep_results.csv). Unless stated otherwise, theory statistics aggregate 270 rows (30 layout instances: 5×5 and 10×10, three densities, five seeds; nine algorithm variants per instance including BFS and each A* configuration). Sweep statistics aggregate 192 rows (5×5 and 10×10, the same three densities, eight constraint labels, four algorithms).

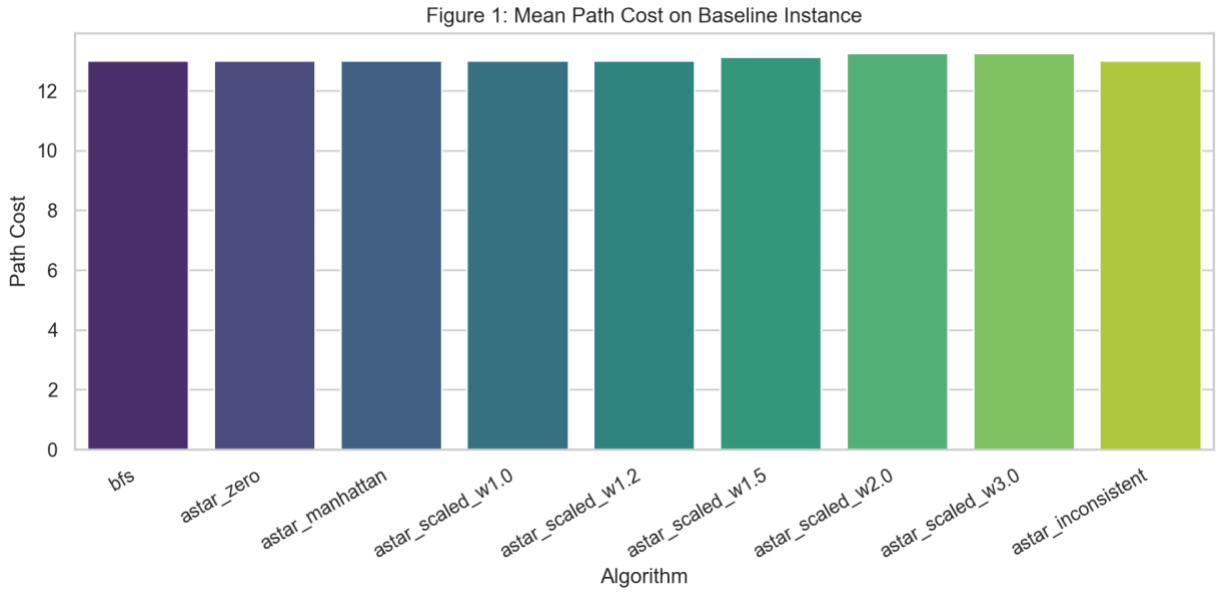

Figure 1: Mean path cost by algorithm on the baseline theory configuration; admissible variants align with the BFS optimal cost while inadmissible weighted settings can depart on larger instances.

Pooling baseline-configuration theory rows, BFS and every admissible A* variant (h = 0, Manhattan, scaled weight 1.0, and the inconsistent heuristic in this log) matched the BFS cost on 100% of runs. For scaled Manhattan with w > 1, optimality declines as expected: w = 1.2 remained 100% optimal in these logs, w = 1.5 fell to 93.3%, and w = 2.0 and w = 3.0 each to 86.7%. Among 120 runs with w > 1, 10 exhibited a positive suboptimality gap; whenever a gap appeared it was 2.0 steps (mean and maximum 2.0). Figure 1 summarizes mean reported path cost by algorithm tag and is most useful read together with the optimality column in the CSV: flat bars indicate agreement with the BFS baseline on the instances in this archive.

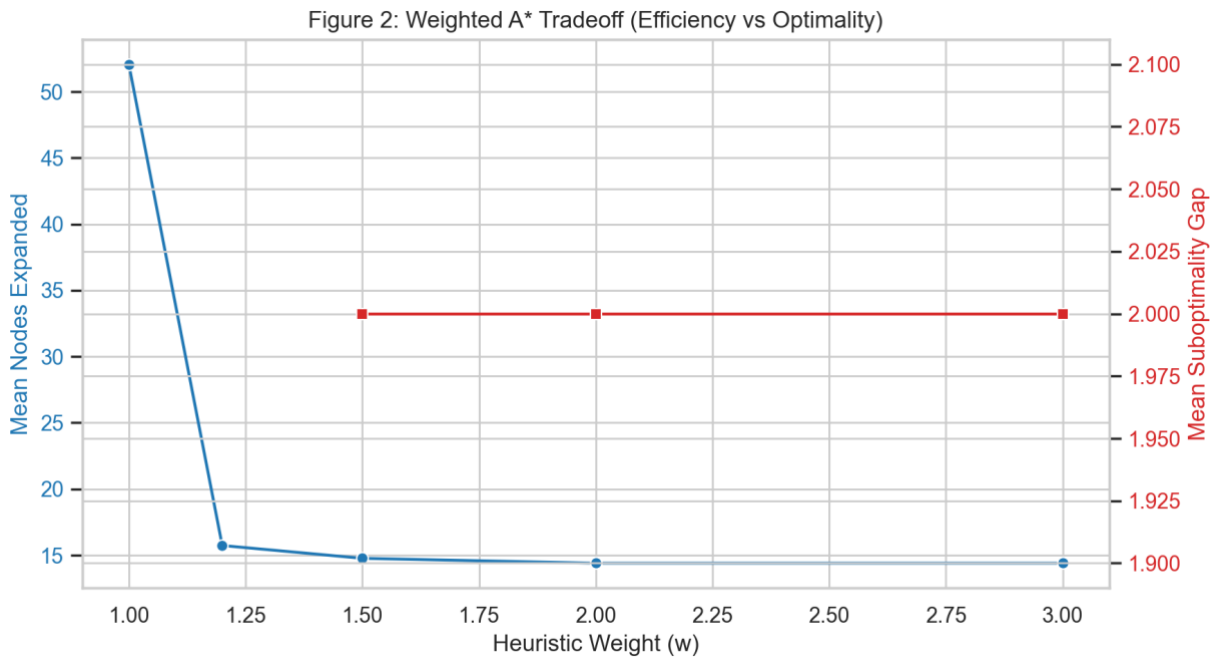

Figure 2: Weighted A*: mean nodes expanded vs heuristic weight (left axis) and mean suboptimality gap (right axis), showing the efficiency–optimality trade-off as w increases.

The same scaled rows show the complementary efficiency story: mean nodes expanded falls from about 52 at w = 1.0 to roughly 16 at w = 1.2 and about 14 for w ∈ {1.5, 2.0, 3.0}, while the mean suboptimality gap grows with w in Figure 2. This pair of trends is the quantitative counterpart to the admissibility discussion in Methods: cheaper search can be purchased with a known optimality risk once w exceeds one.

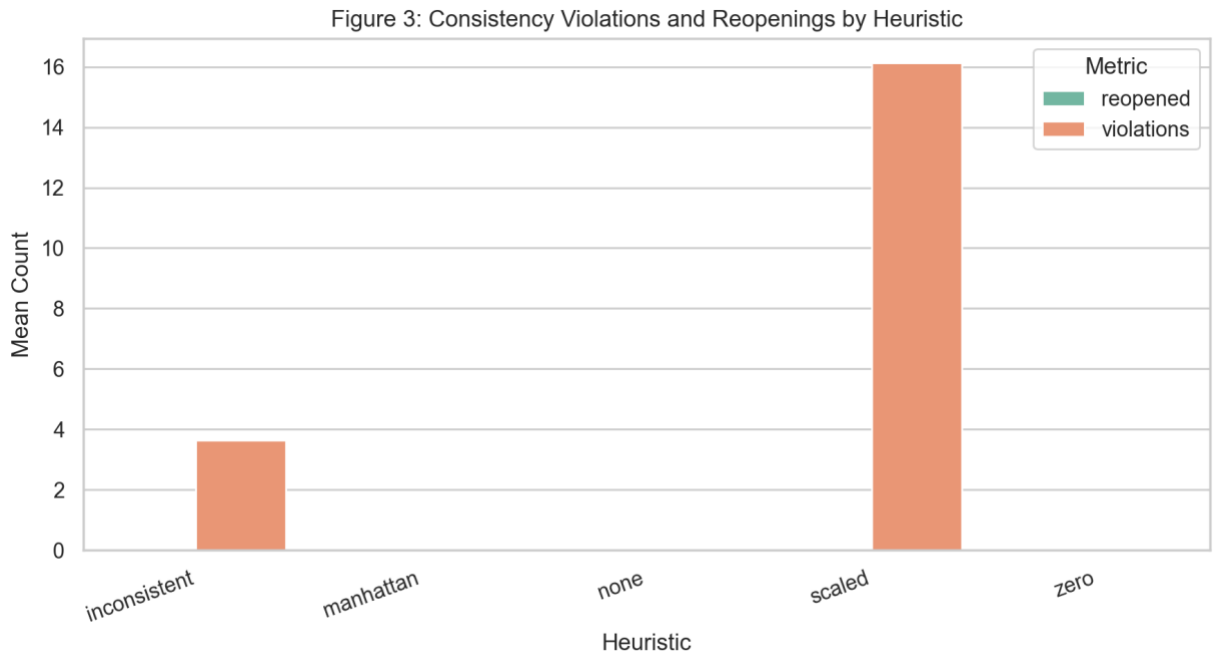

Figure 3: Mean consistency-violation counts and reopenings by heuristic family (theory log).

Consistency-related tallies cluster by heuristic type in Figure 3. The inconsistent construction records non-zero violation counts while remaining optimal on every row in this theory log. Scaled weights above 1.0 register violations in the checker even when no reopenings accumulate in many cells of the log; the plot makes the distinction between “violations observed along edges” and “optimality lost” visible at a glance.

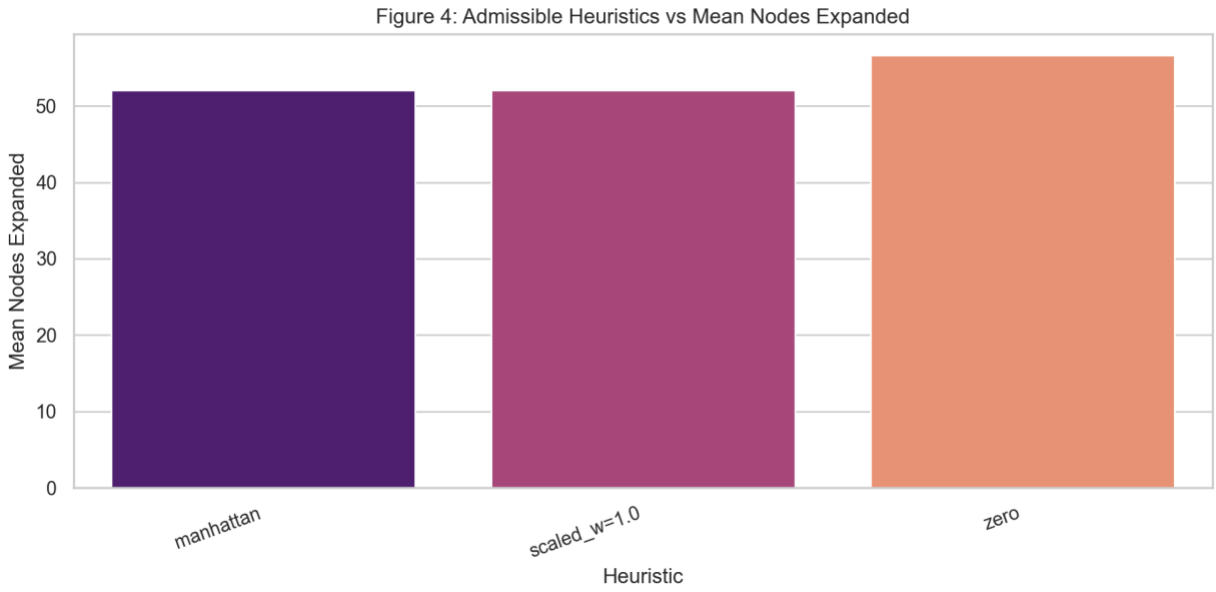

Figure 4: Mean nodes expanded for admissible heuristic choices (h = 0, Manhattan, scaled w ≤ 1).

Among admissible choices, **Manhattan** and **scaled *w* = 1.0** achieve the lowest mean expansions in these logs (about **52**), slightly better on average than ***h* = 0** (uniform-cost-style A*, about **56.6**). **Figure 4** orders exactly these three configurations by mean nodes expanded. The **inconsistent** heuristic also remains admissible in this implementation and averages about **52.1** expansions over the same theory log, but it is summarized in **Figure 3** (consistency violations and reopenings), not in **Figure 4**.

Baseline sweep instance (generated 5×5, obstacle density 0.0, unconstrained configuration). This is not the same map as the fixed default repository maze used by python3 main.py --all; it is the sweep's baseline row on an empty 5×5 draw.

There, BFS and production A* both return optimal path cost 8 with 25 expansions; DFS returns cost 16 in 17 expansions; IDDFS is optimal but uses 928 expansions and about 7.8×10^{-4} s in the saved sweep row—illustrating suboptimality vs optimality vs search effort on a single comparable instance. If the supplementary table from python3 main.py --all / show_results.py is included, it should be labeled as the bundled default layout.

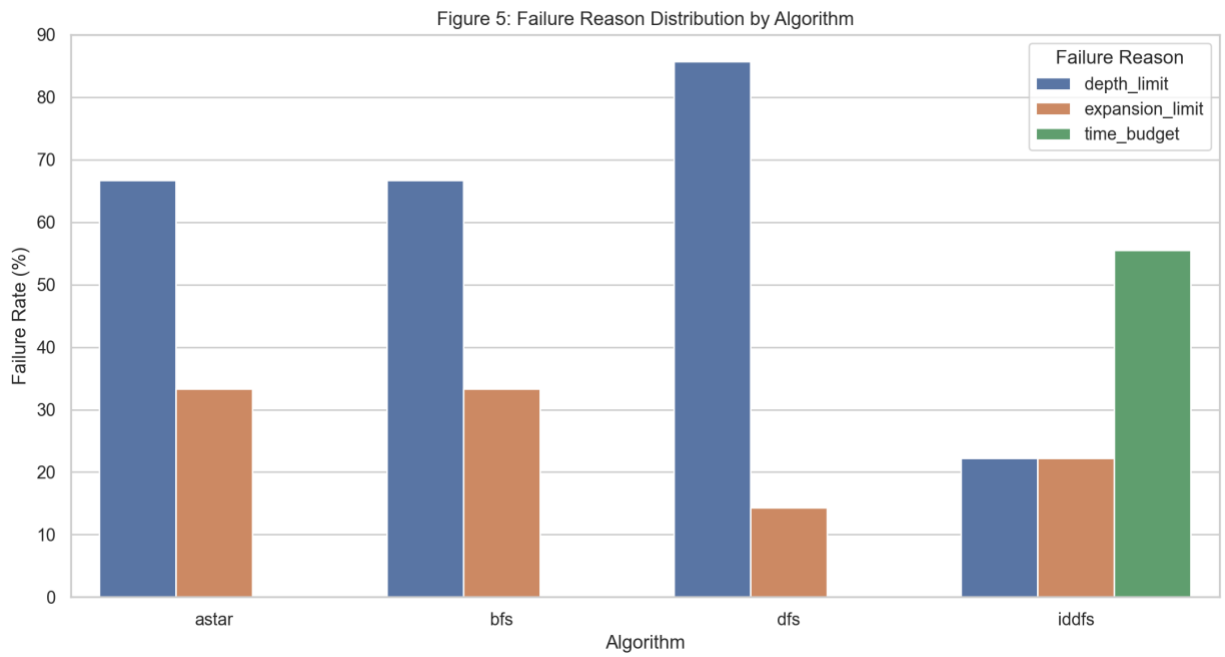

Figure 5: Distribution of failure reasons among unsuccessful sweep runs, by algorithm (percentage within each algorithm's failures)

Under all constraint labels, IDDFS incurred the most failures (27 of 192 rows classified as not finding a solution in the CSV), with BFS and A* tied at 9 and DFS at 7—consistent with iterative deepening struggling most under tight time ceilings and with depth/expansion caps that cut complete algorithms before certification. Figure 5 breaks down failure reasons (for example time_budget, depth_limit, expansion_limit) by algorithm.

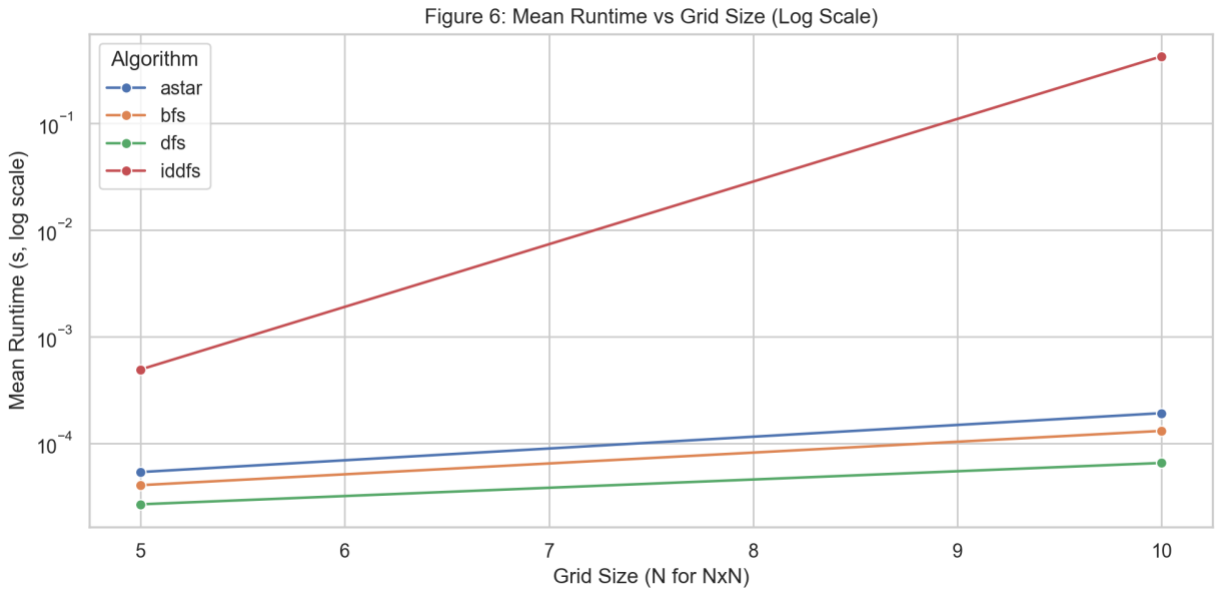

Figure 6: Mean wall time vs grid dimension for each algorithm (baseline sweep configuration only).

For baseline runs only, mean timings stay sub-millisecond for BFS, DFS, and A* while IDDFS averages orders of magnitude longer—driven by repeated deepening stages—and grows sharply from 5×5 means near 10^{-3} s to 10×10 means on the order of seconds in this log. Figure 6 plots mean time against extracted grid dimension for visual comparison.

Automated checks. python3 run_tests.py completed with eight passing tests on the submission snapshot, so the reported behavior is tied to a regression-tested codebase rather than ad hoc notebook exploration.

Conclusions

This project's main strength is **end-to-end clarity**: a small grid domain with **uniform costs** lets classical claims about **BFS optimality**, **A*** with **admissible versus weighted** heuristics, and **consistency monitoring** be turned into **reproducible numbers** (path cost against a BFS baseline, suboptimality gaps, expansions, failure reasons under budgets). The results line up with the standard picture in **Hart, Nilsson, and Raphael's formulation of A* [1]**: **informed A*** with **Manhattan** or **scaled** weights **≤ 1** tracks the **optimal cost** in the logged runs, while **weights above 1** buy **far fewer expansions** at the price of **measurable suboptimality**; the **inconsistent** construction shows how **consistency violations** can be separated from **optimality** on these instances. Compared with **DFS**, which can return **cheap but suboptimal** routes on the same map, and compared with **IDDFS**, which stays **optimal** but **scales badly** in time when deepening is repeated, **BFS** and **A*** occupy the sensible middle ground for **complete, optimal** behavior when resources are generous—while the sweep shows that **any** complete method can fail predictably once **depth, expansion, or time** caps bite, with **IDDFS** especially exposed under **tight time budgets**.

Weaknesses are mostly **domain and engineering scope**, not logical errors. The world is **four-connected, unit-cost, and static**; there is **no** comparison to **Dijkstra** on arbitrary graphs, **JPS**, **Jump Point Search**, **Theta-Star**,

or **hierarchical planning**, which dominate many application settings. **Production A*** in the sweep uses a **single** Manhattan heuristic, whereas the **theory** A* path explores **reopening** and more exotic **h**—two related but not identical code paths. **Runtime** numbers are **small** and **hardware-dependent**; they illustrate **relative** scaling (especially the **IDDFS** curve on **Figure 6**) rather than production benchmarks. Finally, **mean** summaries can **hide** rare failure modes; the **CSV** detail remains important for auditing corner cases.

Looking ahead, **natural extensions** would (i) unify **one** A* core with **pluggable** heuristics and **optional** reopening for all experiments, (ii) add **larger maps**, **non-uniform** terrain costs, **eight-way** movement, or **dynamic** obstacles, (iii) compare against **other** optimal or bounded-suboptimal planners and record **memory** as well as **time**, and (iv) tighten **reproducibility** with **pinned** dependency versions and a single **make**-style script that regenerates **JSON → CSV → figures**. Together with the existing **tests**, those steps would move the work from a **strong course-scale validation** of the **classical A*** perspective **[1]** toward a **more general** pathfinding **evaluation harness**.

WORKS CITED

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Syst. Sci. Cybern.*, vol. SSC-4, no. 2, pp. 100–107, Jul. 1968.

[2] Python Software Foundation, “The Python Language Reference, version 3,” Python Software Foundation, Beaverton, OR, USA, [Online].
Available: https://docs.python.org/3/

[3] The pandas development team, “pandas,” [Online].
Available: https://pandas.pydata.org/

[4] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007.

[5] M. L. Waskom, “seaborn: statistical data visualization,” *J. Open Source Softw.*, vol. 6, no. 60, p. 3021, 2021. [Online].
Available: https://doi.org/10.21105/joss.03021